

code-guide

The sample project from an empty folder to a running Android app. Eleven parts, one file at a time, then the browser loop and the Android build: emulator, real device, signed release.

Every listing below is the file as it exists in the finished project, so nothing here is approximate. Build the parts in order: each one ends with a check that tells you it worked before the next begins.

Part	What it builds	Files
1	Project scaffold and app shell	8
2	Data shapes	3
3	Services	4
4	Login page	5
5	Tabs shell	5
6	Movies catalogue	5
7	Movie detail sub-page	5
8	Menu catalogue	5
9	Menu detail sub-page	5
10	Defunct search tab	5
11	Account page and exit	5

55 files in all, then the Android build.

What you are building

A login page, then a tabs shell with four tabs. Two of the tabs are catalogues of a different genre, and each opens a nested detail route, so the URL reads `tabs/movies/01` rather than opening a pushed top-level page. One tab is a deliberately non-functional search page, and the account tab carries an exit button that returns to login.

The route map the finished app must have

ROUTE MAP

```
app-routing.module.ts
'' -> redirectTo 'login', pathMatch: 'full'
'login' -> loadChildren LoginPageModule
'tabs' -> loadChildren TabsPageModule

tabs-routing.module.ts
'' -> TabsPage, children:
  'movies' -> loadChildren MoviesPageModule
  'menu' -> loadChildren MenuPageModule
  'search' -> loadChildren SearchPageModule
  'account' -> loadChildren AccountPageModule
'' -> redirectTo '/tabs/movies', pathMatch: 'full'

movies-routing.module.ts
'' -> MoviesPage
':id' -> loadChildren MovieDetailPageModule      gives /tabs/movies/01

movie-detail-routing.module.ts
'' -> MovieDetailPage                          empty: the parent took ':id'
```

Before you start

The toolchain

Tool	Version to use	How you know it is ready
Node.js LTS and npm	the LTS release line	<code>node -v, npm -v</code>
Ionic CLI	installed globally	<code>ionic -v</code>
Git	any current release	<code>git --version</code>
VS Code	any current release	the project folder opens in it
Android Studio	Ladybug 2024.2.1 or newer	the SDK shows in the settings
Java (JDK)	21, which ships with Android Studio Ladybug	<code>java -version</code>
Android SDK	compileSdk 35, minSdk 23	Android Studio installs it on first run
Emulator (AVD)	one device created	it appears in Device Manager

Create the project

SCAFFOLD, ONCE

```
# name, template, type: the deck's own form of the command
ionic start appdev-app tabs --type=angular

cd appdev-app
ionic serve          # it should open in a browser before you change anything
```

The two NgModule adjustments

1. Every `@Component` and `@NgModule` needs `standalone: false`, because Angular 19 and later default to `standalone`.
2. `IonicModule` must come from `@ionic/angular/lazy`, not from the root `@ionic/angular` entry point, which is the `standalone` API in Ionic 9. Importing it from the root fails the build with NG8001 on every `ion-*` element.

Test in the browser first. Every part below is checked with `ionic serve` before it is ever pushed to Android, because the browser loop is seconds and the native loop is minutes.

Part 1: Project scaffold and app shell

A blank Ionic Angular project that boots to an empty router outlet.

Files

SRC/MAIN.TS

```
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';

platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch((err) => console.log(err));
```

SRC/APP/APP.MODULE.TS

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { RouteReuseStrategy } from '@angular/router';

import { IonicModule, IonicRouteStrategy } from '@ionic/angular/lazy';

import { AppComponent } from './app.component';
import { AppRoutingModuleModule } from './app-routing.module';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, IonicModule.forRoot(), AppRoutingModuleModule],
  providers: [{ provide: RouteReuseStrategy, useClass: IonicRouteStrategy }],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

SRC/APP/APP.COMPONENT.TS

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.scss'],
  standalone: false,
})
export class AppComponent {
  constructor() {}
}
```

SRC/APP/APP.COMPONENT.HTML

```
<ion-app>
  <ion-router-outlet></ion-router-outlet>
</ion-app>
```

SRC/APP/APP-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { PreloadAllModules, RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  {
    path: '',
    redirectTo: 'login',
    pathMatch: 'full',
  },
  {
    path: 'login',
    loadChildren: () => import('./login/login.module').then(m => m.LoginPageModule),
  },
  {
    path: 'tabs',
    loadChildren: () => import('./tabs/tabs.module').then(m => m.TabsPageModule),
  },
];

@NgModule({
  imports: [RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

SRC/GLOBAL.SCSS

```
@import "@ionic/angular/css/core.css";
@import "@ionic/angular/css/normalize.css";
@import "@ionic/angular/css/structure.css";
@import "@ionic/angular/css/typography.css";
@import "@ionic/angular/css/display.css";
@import "@ionic/angular/css/padding.css";
@import "@ionic/angular/css/float-elements.css";
@import "@ionic/angular/css/text-alignment.css";
@import "@ionic/angular/css/text-transformation.css";
@import "@ionic/angular/css/flex-utils.css";
```

SRC/THEME/VARIABLES.SCSS

```
/* Ionic Variables and Theming. */

:root {
  /* school blue + gold, Lesson 7 Example 1A */
  --ion-color-primary: #0f4c81;
  --ion-color-primary-rgb: 15, 76, 129;
  --ion-color-primary-contrast: #ffffff;
  --ion-color-primary-contrast-rgb: 255, 255, 255;
  --ion-color-primary-shade: #0d4372;
  --ion-color-primary-tint: #275e8e;

  --ion-color-secondary: #f59e0b;
  --ion-color-secondary-rgb: 245, 158, 11;
  --ion-color-secondary-contrast: #111827;
  --ion-color-secondary-contrast-rgb: 17, 24, 39;
  --ion-color-secondary-shade: #d88b0a;
  --ion-color-secondary-tint: #f6a823;
}
```

SRC/APP/APP.COMPONENT.SCSS

```
/* App shell. Deliberately almost empty: the theme values live in
src/theme/variables.scss and the shared helpers in src/global.scss, which is the
three-stylesheet split the L07 lesson teaches. Ionic's page generator also creates
this file, so it is kept for parity with a generated project. */
```

What to watch

- main.ts bootstraps AppModule with platformBrowserDynamic(). No standalone bootstrap here.
- app.module.ts declares AppComponent only, and its imports list carries IonicModule.forRoot() plus AppRoutingModuleModule. Both Ionic names come from @ionic/angular/lazy: in Ionic 9 the root @ionic/angular entry point is the standalone API.

- The `RouteReuseStrategy` provider that maps to `IonicRouteStrategy` is what makes Ionic navigation state survive; without it pages lose their Ionic transition behaviour.
- `app.component.html` is exactly `<ion-app><ion-router-outlet></ion-router-outlet></ion-app>`. Ionic replaces `router-outlet` with its own outlet so pages get stack behaviour.
- Angular 19 and later default to standalone. Because every page here is declared in an `NgModule`, each `@Component` must state `standalone: false` or the build fails with a declaration error.
- In Ionic 9 the root `@ionic/angular` entry point holds the standalone components and `provideIonicAngular()`; the class based `IonicModule` lives in `@ionic/angular/lazy`. Importing `IonicModule` from the root entry point fails the build with NG8001 on every `ion-*` element. `IonicModule` is deprecated in favour of `provideIonicAngular()`, which also works in `NgModule` applications.

Check

`ionic serve` opens a blank shell with no console errors about missing modules.

Source: L01 slide 14 (project structure), L02 slides 3 and 5

Part 2: Data shapes

Interfaces that describe the movie, menu item and account records.

Files

SRC/APP/MODELS/MOVIE.TS

```
export interface Movie {  
  id: number;  
  title: string;  
  posterUrl: string;  
  genre: string;  
  synopsis: string;  
  gallery: string[];  
}
```

SRC/APP/MODELS/MENU-ITEM.TS

```
export interface MenuItem {  
  id: number;  
  name: string;  
  category: string;  
  price: number;  
  stock: number;  
  imageUrl: string;  
  description: string;  
}
```

SRC/APP/MODELS/ACCOUNT.TS

```
export interface Account {  
  avatarUrl: string;  
  fullName: string;  
  username: string;  
  email: string;  
  address: string;  
}
```

What to watch

- These are interface, not class. An interface exists only at compile time and disappears from the shipped JavaScript.
- A model becomes a class only when records need behaviour (methods). Here the records are plain data, so interfaces are the correct choice, which is exactly the distinction L05 slide 9 draws.
- stock: number on MenuItem is what the Menu tab reports as in-stock counts.

Check

No runtime effect. The compiler now rejects `movie.titel`.

Source: L05 slides 3 to 10

Part 3: Services

One service per feature, each injectable at root, holding the data the pages read.

Files

SRC/APP/SERVICES/AUTH.SERVICE.TS

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private loggedIn = false;
  private username = '';

  login(username: string, password: string): boolean {
    // No backend in this activity: one hardcoded credential pair.
    if (username === 'admin' && password === '1234') {
      this.loggedIn = true;
      this.username = username;
      return true;
    }
    return false;
  }

  logout(): void {
    this.loggedIn = false;
    this.username = '';
  }

  isLoggedIn(): boolean {
    return this.loggedIn;
  }

  getUsername(): string {
    return this.username;
  }
}
```

SRC/APP/SERVICES/MOVIE.SERVICE.TS

```

import { Injectable } from '@angular/core';
import { Movie } from '../models/movie';

@Injectable({ providedIn: 'root' })
export class MovieService {
  private movies: Movie[] = [
    {
      id: 1,
      title: 'Iron Man',
      posterUrl: 'assets/movie-posters/iron-man.jpg',
      genre: 'Action',
      synopsis: 'A billionaire weapons engineer builds a suit of armour and becomes a hero.',
      gallery: ['assets/movie-posters/iron-man-1.jpg', 'assets/movie-posters/iron-man-2.jpg'],
    },
    {
      id: 2,
      title: 'Real Steel',
      posterUrl: 'assets/movie-posters/real-steel.jpg',
      genre: 'Action',
      synopsis: 'A father and his son train a scrap robot to fight in the boxing circuit.',
      gallery: ['assets/movie-posters/real-steel-1.jpg', 'assets/movie-posters/real-steel-2.jpg'],
    },
    {
      id: 3,
      title: 'Pacific Rim',
      posterUrl: 'assets/movie-posters/pacific-rim.jpg',
      genre: 'Action',
      synopsis: 'Humanity pilots giant machines against monsters rising from the sea.',
      gallery: ['assets/movie-posters/pacific-rim-1.jpg', 'assets/movie-posters/pacific-rim-2.jpg'],
    },
  ];

  getMovies(): Movie[] {
    return this.movies;
  }

  getMovieById(id: number): Movie | undefined {
    return this.movies.find((movie) => movie.id === id);
  }
}

```

SRC/APP/SERVICES/MENU.SERVICE.TS

```

import { Injectable } from '@angular/core';
import { MenuItem } from '../models/menu-item';

@Injectable({ providedIn: 'root' })
export class MenuService {
  private items: MenuItem[] = [
    {
      id: 1,
      name: 'Chicken Adobo Rice Bowl',
      category: 'Rice Meal',
      price: 129,
      stock: 18,
      imageUrl: 'assets/menu/adobo.jpg',
      description: 'Braised chicken in soy sauce, vinegar, garlic and bay leaf, served over rice.',
    },
    {
      id: 2,
      name: 'Beef Tapa Plate',
      category: 'Rice Meal',
      price: 149,
      stock: 7,
      imageUrl: 'assets/menu/tapa.jpg',
      description: 'Cured beef strips with garlic rice and a fried egg.',
    },
    {
      id: 3,
      name: 'Halo-Halo',
      category: 'Dessert',
      price: 99,
      stock: 0,
      imageUrl: 'assets/menu/halo-halo.jpg',
      description: 'Shaved ice with mixed fruits, beans, leche flan and ube.',
    },
  ];

  getItems(): MenuItem[] {
    return this.items;
  }

  getItemById(id: number): MenuItem | undefined {
    return this.items.find((item) => item.id === id);
  }
}

```

SRC/APP/SERVICES/ACCOUNT.SERVICE.TS

```

import { Injectable } from '@angular/core';
import { Account } from '../models/account';

@Injectable({ providedIn: 'root' })
export class AccountService {
  private account: Account = {
    avatarUrl: 'assets/avatar.png',
    fullName: 'Juan Dela Cruz',
    username: 'juandc',
    email: 'juan.delacruz@example.com',
    address: 'Valenzuela City, Metro Manila, Philippines',
  };

  getAccount(): Account {
    return this.account;
  }
}

```

What to watch

- The decorator is `@Injectable({ providedIn: 'root' })` from `@angular/core`. There is no Service decorator in Angular; a file that imports Service and writes `@Service()` will not compile. This is the single most common copy-paste failure.

- `providedIn: 'root'` registers the service with the root injector, so it is a singleton and never needs an entry in any `@NgModule` providers array.
- The alternative form is `@Injectable()` plus the service added to a module's providers: `[]`. Know both, prefer root for this activity.
- No constructor work beyond storing injected dependencies. L04 slide 6: the constructor creates the object, it does not start the page.
- `getMovieById` returns `Movie | undefined` because `Array.find` can miss. The detail page must tolerate undefined.

Check

Nothing visible yet. A typo in `providedIn` produces a `NullInjectorError` at the first page load.

Source: L04 slides 2 to 6, L05 slides 7 and 16

Part 4: Login page

A form that checks one hardcoded credential pair and routes into the tabs shell.

Files

SRC/APP/LOGIN/LOGIN.PAGE.TS

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.page.html',
  styleUrls: ['./login.page.scss'],
  standalone: false,
})
export class LoginPage {
  username = '';
  password = '';
  errorMessage = '';

  // constructor: wiring only. No navigation, no work with the view.
  constructor(private auth: AuthService, private router: Router) {}

  onLogin(): void {
    if (this.auth.login(this.username, this.password)) {
      this.errorMessage = '';
      this.router.navigate(['/tabs/movies'], { replaceUrl: true });
    } else {
      this.errorMessage = 'Invalid username or password.';
    }
  }
}
```

SRC/APP/LOGIN/LOGIN.PAGE.HTML

```
<ion-content class="ion-padding">
  <div class="login-box">
    <h1>Sign in</h1>
    <p>Applications Development activity: Ionic + Angular, NgModule based.</p>

    <ion-item>
      <ion-label position="stacked">Username</ion-label>
      <ion-input [(ngModel)]="username" placeholder="admin"></ion-input>
    </ion-item>

    <ion-item>
      <ion-label position="stacked">Password</ion-label>
      <ion-input type="password" [(ngModel)]="password" placeholder="1234"></ion-input>
    </ion-item>

    <ion-text color="danger" *ngIf="errorMessage">
      <p>{{ errorMessage }}</p>
    </ion-text>

    <ion-button expand="block" color="primary" (click)="onLogin()">Log in</ion-button>
  </div>
</ion-content>
```

SRC/APP/LOGIN/LOGIN-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { LoginPage } from './login.page';

const routes: Routes = [
  {
    path: '',
    component: LoginPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class LoginPageRoutingModule {}
```

SRC/APP/LOGIN/LOGIN.MODULE.TS

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { LoginPageRoutingModule } from './login-routing.module';

import { LoginPage } from './login.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    LoginPageRoutingModule
  ],
  declarations: [LoginPage]
})
export class LoginPageModule {}
```

SRC/APP/LOGIN/LOGIN.PAGE.SCSS

```
/* Login. The only page-specific styling needed: keep the form narrow so it does not
stretch across a desktop browser window. */
.login-box {
  max-width: 420px;
  margin: 0 auto;
}
```

What to watch

- The page's own routing module declares { path: '', component: LoginPage } and exports RouterModule. Note forChild, never forRoot, outside the app routing module.
- login.module.ts imports CommonModule, FormsModule, IonicModule and the page routing module, then declares the page. FormsModule is what enables [(ngModel)]. IonicModule is imported from @ionic/angular/lazy, not from the root entry point.
- [(ngModel)] is the two-way banana-in-a-box binding. Without FormsModule in the module imports the template throws Can't bind to 'ngModel'.
- Redirect with code, not with markup: this.router.navigate(['/tabs/movies']). The destination depends on a function result, which is exactly when L02 slide 9 says to use TypeScript navigation.
- The constructor only wires AuthService and Router. Validation lives in a method, and nothing page-related runs before the user taps.

Check

Submitting admin / 1234 changes the URL to /tabs/movies; anything else prints the error line.

Source: L02 slide 9, L04 slides 6 to 8, L06 slide 13

Part 5: Tabs shell

The tab bar and the four child routes it switches between.

Files

SRC/APP/TABS/TABS.PAGE.TS

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-tabs',
  templateUrl: './tabs.page.html',
  styleUrls: ['./tabs.page.scss'],
  standalone: false,
})
export class TabsPage {
  constructor() {}
}
```

SRC/APP/TABS/TABS.PAGE.HTML

```
<ion-tabs>
  <ion-tab-bar slot="bottom">
    <ion-tab-button tab="movies" href="/tabs/movies">
      <ion-icon name="film-outline"></ion-icon>
      <ion-label>Movies</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="menu" href="/tabs/menu">
      <ion-icon name="restaurant-outline"></ion-icon>
      <ion-label>Menu</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="search" href="/tabs/search">
      <ion-icon name="search-outline"></ion-icon>
      <ion-label>Search</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="account" href="/tabs/account">
      <ion-icon name="person-outline"></ion-icon>
      <ion-label>Account</ion-label>
    </ion-tab-button>
  </ion-tab-bar>
</ion-tabs>
```

SRC/APP/TABS/TABS-ROUTING.MODULE.TS

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { TabsPage } from './tabs.page';

const routes: Routes = [
  {
    path: '',
    component: TabsPage,
    children: [
      {
        path: 'movies',
        loadChildren: () => import('../movies/movies.module').then(m => m.MoviesPageModule),
      },
      {
        path: 'menu',
        loadChildren: () => import('../menu/menu.module').then(m => m.MenuPageModule),
      },
      {
        path: 'search',
        loadChildren: () => import('../search/search.module').then(m => m.SearchPageModule),
      },
      {
        path: 'account',
        loadChildren: () => import('../account/account.module').then(m => m.AccountPageModule),
      },
      {
        path: '',
        redirectTo: '/tabs/movies',
        pathMatch: 'full',
      },
    ],
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class TabsPageRoutingModule {}

```

SRC/APP/TABS/TABS.MODULE.TS

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { TabsPageRoutingModule } from './tabs-routing.module';

import { TabsPage } from './tabs.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    TabsPageRoutingModule
  ],
  declarations: [TabsPage]
})
export class TabsPageModule {}

```

SRC/APP/TABS/TABS.PAGE.SCSS

```

/* Tabs shell. Nothing page-specific is needed here: the tab bar is styled entirely
through the Ionic theme, and its colour comes from --ion-color-primary in
src/theme/variables.scss. */

```

What to watch

- children: inside the tabs route is the whole mechanism. Each entry pairs a path with a lazy loadChildren that resolves to a page module, not a page class.
- loadChildren: `() => import('../movies/movies.module').then(m => m.MoviesPageModule)` is the NgModule form. The standalone form shown in the deck is `loadComponent: () => import('../movies/movies.page').then(m => m.MoviesPage)`, and it will not match this project.
- Each ion-tab-button needs `tab="..."` and `href="/tabs/..."`. The tab attribute is what Ionic uses to match the button to the active child route.
- The final child `{ path: '', redirectTo: '/tabs/movies', pathMatch: 'full' }` is what makes bare /tabs land on the first tab. Keep `pathMatch: 'full'` or an empty path swallows every URL.
- Tab pages keep their own history stack. L02 slide 13: with tabs, browser back is not always app back.

Check

Navigating to /tabs shows four tab buttons and lands on Movies.

Source: L02 slides 13 to 14

Part 6: Movies catalogue

A list of movies where each row opens a nested detail route.

Files

SRC/APP/MOVIES/MOVIES.PAGE.TS

```
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Movie } from '../models/movie';
import { MovieService } from '../services/movie.service';

@Component({
  selector: 'app-movies',
  templateUrl: './movies.page.html',
  styleUrls: ['./movies.page.scss'],
  standalone: false,
})
export class MoviesPage implements OnInit {
  movies: Movie[] = [];

  // constructor: dependency wiring only.
  constructor(private movieService: MovieService, private router: Router) {}

  // ngOnInit: the first place allowed to touch the page data.
  ngOnInit(): void {
    this.movies = this.movieService.getMovies();
  }

  openMovie(id: number): void {
    this.router.navigate(['tabs/movies', id]);
  }
}
```

SRC/APP/MOVIES/MOVIES.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Movies</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-list>
    <ion-item *ngFor="let movie of movies" button (click)="openMovie(movie.id)">
      <ion-thumbnail slot="start">
        <img [src]="movie.posterUrl" [alt]="movie.title" />
      </ion-thumbnail>
      <ion-label>
        <h2>{{ movie.title }}</h2>
        <p>{{ movie.genre }}</p>
      </ion-label>
      <ion-note slot="end">{{ movie.id }}</ion-note>
    </ion-item>
  </ion-list>
</ion-content>
```

SRC/APP/MOVIES/MOVIES-ROUTING.MODULE.TS

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MoviesPage } from './movies.page';

const routes: Routes = [
  {
    path: '',
    component: MoviesPage,
  },
  {
    // the catalogue owns its own detail segment, so the URL is /tabs/movies/01
    path: ':id',
    loadChildren: () =>
      import('../movie-detail/movie-detail.module').then(m => m.MovieDetailPageModule),
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MoviesPageRoutingModule {}

```

SRC/APP/MOVIES/MOVIES.MODULE.TS

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { MoviesPageRoutingModule } from './movies-routing.module';

import { MoviesPage } from './movies.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    MoviesPageRoutingModule
  ],
  declarations: [MoviesPage]
})
export class MoviesPageModule {}

```

SRC/APP/MOVIES/MOVIES.PAGE.SCSS

```

/* Movies list. Page-specific tuning only, and through Ionic's own settings rather
   than its internal class names, which is the L07 rule. */
ion-thumbnail {
  --size: 56px;
  --border-radius: 6px;
}

```

What to watch

- `movies-routing.module.ts` holds two entries: `{ path: '', component: MoviesPage }` for the list and `{ path: ':id', loadChildren: ... }` for the detail module. This is what produces the URL shape `/tabs/movies/01` rather than pushing a new top-level page.
- Because the parent route in `tabs-routing.module.ts` names only `movies`, the `:id` segment lives here. The `movies` feature therefore owns its own detail segment, which is why the routes stay readable.
- The tap handler navigates with `this.router.navigate(['/tabs/movies', id])`. The array form is the correct way to pass a route parameter; a single concatenated string is the common mistake.
- `ngOnInit` assigns `this.movies = this.movieService.getMovies()`. The constructor stays a wiring line. L04 slide 7 is exactly this split.

- `*ngFor` needs the element to be inside a declared component of this module, and `CommonModule` in the module imports is what supplies it.

Check

`/tabs/movies` lists three titles. Tapping the first changes the URL to `/tabs/movies/1`.

Source: L02 slides 10 and 14, L04 slides 6 to 8, L05 slide 13

Part 7: Movie detail sub-page

`/tabs/movies/01` renders one movie and offers an Ionic back button.

Files

SRC/APP/MOVIE-DETAIL/MOVIE-DETAIL.PAGE.TS

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { Movie } from '../models/movie';
import { MovieService } from '../services/movie.service';

@Component({
  selector: 'app-movie-detail',
  templateUrl: './movie-detail.page.html',
  styleUrls: ['./movie-detail.page.scss'],
  standalone: false,
})
export class MovieDetailPage implements OnInit {
  movie?: Movie;

  constructor(private route: ActivatedRoute, private movieService: MovieService) {}

  ngOnInit(): void {
    // Route parameters always arrive as strings.
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.movie = this.movieService.getMovieById(id);
  }
}
```

SRC/APP/MOVIE-DETAIL/MOVIE-DETAIL.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/tabs/movies"></ion-back-button>
    </ion-buttons>
    <ion-title>{{ movie?.title }}</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding" *ngIf="movie">
  <img [src]="movie.posterUrl" [alt]="movie.title" class="detail-poster" />
  <h1>{{ movie.title }}</h1>
  <p class="detail-genre">{{ movie.genre }}</p>
  <p>{{ movie.synopsis }}</p>

  <h3>Gallery</h3>
  <img *ngFor="let shot of movie.gallery" [src]="shot" class="detail-shot" />
</ion-content>

<ion-content class="ion-padding" *ngIf="!movie">
  <ion-button expand="block" color="medium">
    No movie found for this id.
  </ion-button>
</ion-content>
```

SRC/APP/MOVIE-DETAIL/MOVIE-DETAIL-ROUTING.MODULE.TS

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MovieDetailPage } from './movie-detail.page';

const routes: Routes = [
  {
    // the parent route already carried /movies/:id, so this module sits at ''
    path: '',
    component: MovieDetailPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MovieDetailPageRoutingModule {}

```

SRC/APP/MOVIE-DETAIL/MOVIE-DETAIL.MODULE.TS

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { MovieDetailPageRoutingModule } from './movie-detail-routing.module';

import { MovieDetailPage } from './movie-detail.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    MovieDetailPageRoutingModule
  ],
  declarations: [MovieDetailPage]
})
export class MovieDetailPageModule {}

```

SRC/APP/MOVIE-DETAIL/MOVIE-DETAIL.PAGE.SCSS

```

/* Movie detail. The poster takes the full width of the content and the gallery shots
   sit in a row. */
.detail-poster {
  width: 100%;
  display: block;
  border-radius: 10px;
}

.detail-genre {
  color: var(--ion-color-medium);
  text-transform: uppercase;
  font-size: 0.8rem;
  letter-spacing: 0.04em;
}

.detail-shot {
  width: 31%;
  margin-right: 2%;
  border-radius: 8px;
}

```

What to watch

- This module's own route is { path: '', component: MovieDetailPage }, not :id. The parent already consumed /movies/:id, so nothing is left for this module to match. Copying the reference repo here (which puts :id in the child module because its parent path is just movie-detail) produces a dead route.

- Read the parameter in `ngOnInit`, never in the constructor: `Number(this.route.snapshot.paramMap.get('id'))`. Route parameters always arrive as strings, so `Number(...)` or `+` is required before comparing with `movie.id`.
- `paramMap` includes parameters inherited from ancestor routes, which is why `'id'` is visible here even though the path segment belongs to the parent route.
- `movie?: Movie` plus `*ngIf="movie"` on the content block guards the template against an id that matches nothing. Without it, `{{ movie.title }}` throws on an empty record.
- `<ion-back-button defaultHref="/tabs/movies"></ion-back-button>` gives the Ionic back behaviour. `defaultHref` is the fallback when there is no previous page in the stack, which is exactly the case when the URL is opened directly.

Check

Opening `/tabs/movies/1` directly still renders the movie and the back button returns to the list.

Source: L02 slides 10 to 12, L03 slides 6 and 9

Part 8: Menu catalogue

The second genre tab: a food menu that reports how many are in stock.

Files

SRC/APP/MENU/MENU.PAGE.TS

```
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { MenuItem } from '../models/menu-item';
import { MenuService } from '../services/menu.service';

@Component({
  selector: 'app-menu',
  templateUrl: './menu.page.html',
  styleUrls: ['./menu.page.scss'],
  standalone: false,
})
export class MenuPage implements OnInit {
  items: MenuItem[] = [];

  constructor(private menuService: MenuService, private router: Router) {}

  ngOnInit(): void {
    this.items = this.menuService.getItems();
  }

  openItem(id: number): void {
    this.router.navigate(['tabs/menu', id]);
  }

  stockLabel(item: MenuItem): string {
    return item.stock > 0 ? item.stock + ' in stock' : 'Out of stock';
  }
}
```

SRC/APP/MENU/MENU.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="secondary">
    <ion-title>Menu</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-list>
    <ion-item *ngFor="let item of items" button (click)="openItem(item.id)">
      <ion-thumbnail slot="start">
        <img [src]="item.imageUrl" [alt]="item.name" />
      </ion-thumbnail>
      <ion-label>
        <h2>{{ item.name }}</h2>
        <p>{{ item.category }}</p>
        <p>{{ stockLabel(item) }}</p>
      </ion-label>
      <ion-note slot="end">PHP {{ item.price }}</ion-note>
    </ion-item>
  </ion-list>
</ion-content>
```

SRC/APP/MENU/MENU-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MenuPage } from './menu.page';

const routes: Routes = [
  {
    path: '',
    component: MenuPage,
  },
  {
    path: ':id',
    loadChildren: () =>
      import('../menu-detail/menu-detail.module').then(m => m.MenuDetailPageModule),
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MenuPageRoutingModule {}
```

SRC/APP/MENU/MENU.MODULE.TS

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { MenuPageRoutingModule } from './menu-routing.module';

import { MenuPage } from './menu.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    MenuPageRoutingModule
  ],
  declarations: [MenuPage]
})
export class MenuPageModule {}
```

SRC/APP/MENU/MENU.PAGE.SCSS

```
/* Menu list. Nothing page-specific beyond the theme: the same list pattern as the
   movies catalogue, with the stock count shown through ion-note. */
```

What to watch

- Identical structure to the movies feature, with menu and menu/:id in place of movies and movies/:id. The repetition is the point: once the pattern is understood, the second catalogue is mechanical.
- stockLabel(item) is a component method called from the template. Methods in templates run on every change-detection pass, so keep them cheap and side-effect free.
- {{ item.stock > 0 ? item.stock + ' in stock' : 'Out of stock' }} would also work inline, but the method version keeps the template readable, which is the L07 slide 20 preference.

Check

/tabs/menu lists the food records and shows 7 in stock and Out of stock on the right rows.

Source: L02 slide 14, L05 slide 13, L07 slide 20

Part 9: Menu detail sub-page

`/tabs/menu/01` renders one menu item with its stock.

Files

SRC/APP/MENU-DETAIL/MENU-DETAIL.PAGE.TS

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { MenuItem } from '../models/menu-item';
import { MenuService } from '../services/menu.service';

@Component({
  selector: 'app-menu-detail',
  templateUrl: './menu-detail.page.html',
  styleUrls: ['./menu-detail.page.scss'],
  standalone: false,
})
export class MenuDetailPage implements OnInit {
  item?: MenuItem;

  constructor(private route: ActivatedRoute, private menuService: MenuService) {}

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.item = this.menuService.getItemById(id);
  }
}
```

SRC/APP/MENU-DETAIL/MENU-DETAIL.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar color="secondary">
    <ion-buttons slot="start">
      <ion-back-button defaultHref="/tabs/menu"></ion-back-button>
    </ion-buttons>
    <ion-title>{{ item?.name }}</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding" *ngIf="item">
  <img [src]="item.imageUrl" [alt]="item.name" class="detail-image" />
  <h1>{{ item.name }}</h1>
  <p>{{ item.category }}</p>
  <p>{{ item.description }}</p>

  <ion-list>
    <ion-item>
      <ion-label>Price</ion-label>
      <ion-note slot="end">PHP {{ item.price }}</ion-note>
    </ion-item>
    <ion-item>
      <ion-label>Stock</ion-label>
      <ion-note slot="end" [color]="item.stock > 0 ? 'success' : 'danger'">{{ item.stock }}</ion-note>
    </ion-item>
  </ion-list>
</ion-content>
```

SRC/APP/MENU-DETAIL/MENU-DETAIL-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MenuDetailPage } from './menu-detail.page';

const routes: Routes = [
  {
    path: '',
    component: MenuDetailPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MenuDetailPageRoutingModule {}
```

SRC/APP/MENU-DETAIL/MENU-DETAIL.MODULE.TS

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { MenuDetailPageRoutingModule } from './menu-detail-routing.module';

import { MenuDetailPage } from './menu-detail.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    MenuDetailPageRoutingModule
  ],
  declarations: [MenuDetailPage]
})
export class MenuDetailPageModule {}
```

SRC/APP/MENU-DETAIL/MENU-DETAIL.PAGE.SCSS

```
/* Menu detail. The dish image gets the same treatment as the movie poster, so the two
   catalogues stay visually symmetric. */
.detail-image {
  width: 100%;
  display: block;
  border-radius: 10px;
}
```

What to watch

- Same four files, same { path: '' } route, same ngOnInit parameter read. The only difference is MenuService and defaultHref="/tabs/menu".
- [color]="item.stock > 0 ? 'success' : 'danger'" uses a property binding to pick an Ionic theme colour from data. This is the L07 slide 10 pattern: the template asks for a colour name, the theme supplies the value.

Check

/tabs/menu/3 shows stock 0 in danger red.

Source: L07 slides 8 to 10

Part 10: Defunct search tab

A search bar that renders and binds but filters nothing.

Files

SRC/APP/SEARCH/SEARCH.PAGE.TS

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-search',
  templateUrl: './search.page.html',
  styleUrls: ['./search.page.scss'],
  standalone: false,
})
export class SearchPage {
  searchTerm = '';

  constructor() {}

  // Intentionally defunct: the bar renders and binds, but nothing filters.
  // The activity only requires the control to exist on its own tab.
  onSearch(): void {}
}
```

SRC/APP/SEARCH/SEARCH.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>Search</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-searchbar [(ngModel)]="searchTerm" (ionInput)="onSearch()" placeholder="Search (not wired)"></ion-searchbar>

  <ion-note color="medium">
    This search bar is deliberately non-functional.
  </ion-note>
</ion-content>
```

SRC/APP/SEARCH/SEARCH-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { SearchPage } from './search.page';

const routes: Routes = [
  {
    path: '',
    component: SearchPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class SearchPageRoutingModule {}
```

SRC/APP/SEARCH/SEARCH.MODULE.TS

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { SearchPageRoutingModule } from './search-routing.module';

import { SearchPage } from './search.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    SearchPageRoutingModule
  ],
  declarations: [SearchPage]
})
export class SearchPageModule {}

```

SRC/APP/SEARCH/SEARCH.PAGE.SCSS

```

/* Search tab. Deliberately defunct: the searchbar renders and binds through ngModel,
and filters nothing. No page-specific styling is needed. */

```

What to watch

- `<ion-searchbar [(ngModel)]="searchTerm" (ionInput)="onSearch()"></ion-searchbar>`. The bar binds, so the control is real, but `onSearch()` is empty by design.
- Do not add filtering. The requirement is a non-functional control on its own tab, and adding logic would change the deliverable.
- `ion-searchbar` is not covered by any deck, so this page is written from the component reference rather than from a lesson. Note that gap explicitly if asked where a given line came from.

Check

Typing in the bar echoes nothing and the list never appears.

Source: no lesson source (see CLASS_NOTES.md gaps)

Part 11: Account page and exit

The account record plus an exit button that returns to login.

Files

SRC/APP/ACCOUNT/ACCOUNT.PAGE.TS

```
import { Component, OnInit } from '@angular/core';
import { NavController } from '@ionic/angular/lazy';
import { Account } from '../models/account';
import { AccountService } from '../services/account.service';
import { AuthService } from '../services/auth.service';

@Component({
  selector: 'app-account',
  templateUrl: './account.page.html',
  styleUrls: ['./account.page.scss'],
  standalone: false,
})
export class AccountPage implements OnInit {
  account?: Account;

  constructor(
    private accountService: AccountService,
    private auth: AuthService,
    private navCtrl: NavController
  ) {}

  ngOnInit(): void {
    this.account = this.accountService.getAccount();
  }

  exit(): void {
    // clear the session first, then reset the stack so Back cannot return here
    this.auth.logout();
    this.navCtrl.navigateRoot('/login', { replaceUrl: true });
  }
}
```

SRC/APP/ACCOUNT/ACCOUNT.PAGE.HTML

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>Account</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding" *ngIf="account">
  <div class="account-card">
    <img [src]="account.avatarUrl" [alt]="account.fullName" class="account-avatar" />
    <h2>{{ account.fullName }}</h2>
    <p>&#64;{{ account.username }}</p>
  </div>

  <ion-list>
    <ion-item>
      <ion-label>Email</ion-label>
      <ion-note slot="end">{{ account.email }}</ion-note>
    </ion-item>
    <ion-item>
      <ion-label>Address</ion-label>
      <ion-note slot="end">{{ account.address }}</ion-note>
    </ion-item>
  </ion-list>

  <ion-button expand="block" color="danger" (click)="exit()">Exit to login</ion-button>
</ion-content>
```

SRC/APP/ACCOUNT/ACCOUNT-ROUTING.MODULE.TS

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { AccountPage } from './account.page';

const routes: Routes = [
  {
    path: '',
    component: AccountPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class AccountPageRoutingModule {}
```

SRC/APP/ACCOUNT/ACCOUNT.MODULE.TS

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

import { IonicModule } from '@ionic/angular/lazy';

import { AccountPageRoutingModule } from './account-routing.module';
import { AccountPage } from './account.page';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    AccountPageRoutingModule
  ],
  declarations: [AccountPage]
})
export class AccountPageModule {}
```

SRC/APP/ACCOUNT/ACCOUNT.PAGE.SCSS

```
/* Account. The avatar is round and the details sit in a card. */
.account-card {
  text-align: center;
  padding: 12px 0 4px;
}

.account-avatar {
  width: 88px;
  height: 88px;
  border-radius: 50%;
  object-fit: cover;
}
```

What to watch

- `exit()` does two things in order: `this.auth.logout()` clears the session, then `this.navCtrl.navigateRoot('/login', { replaceUrl: true })` returns to the login page.
- Two options matter here. `navigateRoot` sets the transition to a root move instead of a forward push, and `replaceUrl` replaces the history entry being left, so Back cannot return to the tabs once the session is cleared. `navigateRoot` on its own only sets the direction and then navigates.
- `NavController` comes from `@ionic/angular` and is injected in the constructor like any other dependency. This is the L02 slide 9 second option, and it is the right tool when the destination is a stack reset.

Check

Exit returns to the login page, and the browser Back button does not re-open the tabs.

Source: L02 slides 9 and 12, L04 slide 6

Run it in the browser

THE LOOP YOU WILL LIVE IN

```
ionic serve      # runs the app with live reload  
ionic build      # compiles the production web assets into www/
```

Walk it before touching Android: sign in, move through the four tabs, open a catalogue entry and its detail page, press back, and use the exit button. The search tab is inert by design; that is the exercise, not a defect. If the exit button leaves the tabs reachable with Back, the `{ replaceUrl: true }` option is missing.

Build it as an Android app

The web build is the app: Capacitor wraps `www/` in a native project. Capacitor 7 requires Android Studio Ladybug 2024.2.1 or newer and Java 21, which ships with it; the Android SDK it installs compiles against SDK 35 with a minimum of SDK 23.

One-time setup

- Install Android Studio Ladybug 2024.2.1 or newer. It brings the JDK 21 that Capacitor 7 expects, so no separate Java install is needed.
- Let it install the Android SDK on first run, then open **Device Manager** and create one virtual device (an AVD). The deck's environment variable `ANDROID_SDK_ROOT` points the tools at that SDK; Android Studio also writes the same path into `android/local.properties` as `sdk.dir`.
- For a physical phone: enable Developer Options (tap Build number seven times), then USB debugging, then connect it and accept the RSA prompt. `adb` devices should list it.

Add the Android platform

- `ionic build` compiles the web assets into `www/`.
- `ionic cap add android` creates the native project in `android/`. Run it once.
- `ionic cap sync` copies `www/` into the native project and applies plugin changes. Run it after every web build. This is the deck's `cap sync`.

Run it on the emulator (virtual)

- `ionic cap open android` opens the native project in Android Studio; pick the emulator in the device dropdown and press Run.
- Or from the terminal: `ionic cap run android` lists the available targets and builds and installs to the one you choose.
- The first build takes minutes while Gradle downloads its dependencies. Later builds are quick.

Run it on a real device (physical)

- With USB debugging on and the phone connected, `adb devices` lists it.
- `ionic cap run android --target <device-id>` builds, installs and launches on that device. Android Studio's Run button does the same with the phone selected.
- If the app opens blank on a device, the web assets are stale: run `ionic build` then `ionic cap sync` and install again.

Build the signed release

- `ionic build` then `ionic cap sync` first, so the release carries the current web assets.
- Create a keystore once, in Android Studio under **Build > Generate Signed Bundle / APK**, or with `keytool`. Keep the keystore and its password: an update to a published app must be signed with the same one.
- For the Play Store, generate an **Android App Bundle** (`.aab`); for direct installation, an **APK**. The wizard writes the signed files into `android/app/build/outputs/bundle/release/` and `.../apk/release/`.
- From the terminal instead: `cd android` then `gradlew.bat bundleRelease` (AAB) or `gradlew.bat assembleRelease` (APK), once signing is configured in `android/app/build.gradle`.

The commands, end to end

FROM WEB BUILD TO INSTALLED APP

```

ionic build                # web assets into www/
ionic cap add android      # once: creates android/
ionic cap sync             # after every build: copies www/ in, applies plugins
ionic cap open android     # opens Android Studio; press Run for emulator or device
# or, from the terminal:
ionic cap run android      # pick the emulator or the connected device

# signed release, from the terminal:
cd android
gradlew.bat bundleRelease  # .aab for the Play Store
gradlew.bat assembleRelease # .apk for direct installation

```

Where the signed files land

Output	Path	Use it for
Android App Bundle	android/app/build/outputs/bundle/release/app-release.aab	the Play Store
APK	android/app/build/outputs/apk/release/app-release.apk	installing directly, or sharing the file

When it goes wrong

Every entry below was hit and diagnosed while building this project.

Symptom	Cause	Fix
NG8001 on every ion-* element	IonicModule imported from the root @ionic/angular entry point, which is the standalone API in Ionic 9	import it from @ionic/angular/lazy, in app.module.ts and every page module
A declaration error naming a component	Angular 19 and later default to standalone	add standalone: false to that component's @Component
A blank shell, or no page transitions	the routed pages are rendered by plain router-outlet	app.component.html is <ion-app><ion-router-outlet></ion-router-outlet></ion-app>
NullInjectorError at page load	the service is not reachable from the injector the page resolves from	check the module's providers and imports first, before reading the service itself
A list does not refresh when you come back to it	the load sits in ngOnInit, which runs once	move it to ionViewWillEnter, which runs on every entry
Back re-enters the tabs after logout	navigateRoot sets the transition but leaves the history entry	pass { replaceUrl: true } to navigateRoot, and to the login navigation
[(ngModel)] does not compile	FormsModule is missing	import FormsModule in that page's module
A poster image 404s	the asset URL is relative	use the root relative form: assets/movie-posters/iron-man.jpg